

DOKUMENTACJA PROJEKTOWA

Temat: Aplikacja Webowa dla Kliniki Weterynaryjnej "DracoVet"

Przedmiot: Technologie Internetu (TIN)

Autor: Laura Barańska, s30075

1. Opis Projektu

Celem projektu było stworzenie aplikacji webowej typu Single Page Application (SPA) do kompleksowego zarządzania kliniką weterynaryjną. System umożliwia interakcję dwóch grup użytkowników: Klientów (właścicieli zwierząt) oraz Personelu Medycznego (Weterynarzy). Aplikacja realizuje funkcjonalność zarówno po stronie serwera (backend), jak i przeglądarki (frontend). Kluczowe cechy systemu to wielojęzyczność (i18n), bezpieczne uwierzytelnianie oparte o tokeny (JWT) oraz dynamiczne dostosowywanie interfejsu w zależności od roli zalogowanego użytkownika.

2. Wykorzystane Technologie

Aplikacja została zbudowana w architekturze Klient-Serwer przy użyciu następujących technologii:

- **Baza danych:** MySQL - MAMP.
- **Backend:** Node.js + Express.js.
- **Frontend:** React.js + React Router (SPA).
- **Biblioteki i narzędzia:**
 - mysql2: komunikacja z bazą danych.
 - bcrypt: bezpieczne hashowanie haseł użytkowników.
 - jsonwebtoken (JWT): obsługa autoryzacji
 - i18next: obsługa tłumaczeń (Polski / Angielski).
 - axios: asynchroniczna komunikacja z API.

3. Instrukcja Uruchomienia

Poniżej przedstawiono kroki niezbędne do uruchomienia aplikacji na lokalnym środowisku:

Krok 1: Konfiguracja Bazy Danych

1. Uruchom serwer MySQL (np. poprzez MAMP).
2. Utwórz nową bazę danych (domyślna nazwa w konfiguracji to „klinika”).

3. Zaimportuj dostarczony skrypt SQL, który tworzy 5 tabel (Users, Animal, Service, Appointment, Appointment_Service) i wypełnia je danymi startowymi (usługami).
4. W pliku server/.env dostosuj dane logowania (host, user, password, port) do swojego środowiska.

Krok 2: Uruchomienie Serwera (Backend)

1. W terminalu przejdź do katalogu serwera: `cd server`
2. Zainstaluj zależności: `npm install`
3. Uruchom serwer: `node server.js` (Serwer uruchomi się domyślnie na porcie 3001).

Krok 3: Uruchomienie Klienta (Frontend)

1. W nowym oknie terminala przejdź do katalogu klienta: `cd client`
2. Zainstaluj zależności: `npm install`
3. Uruchom aplikację: `npm start`

Aplikacja otworzy się automatycznie pod adresem `http://localhost:3000`.

4. Wykaz Funkcjonalności i Kontrola Dostępu

System implementuje ścisłą kontrolę dostępu opartą na rolach (client vs vet).

A. Użytkownik Niezalogowany (Gość):

- Dostęp do strony głównej (informacje o klinice).
- Rejestracja konta (z wyborem roli: Klient lub Weterynarz).
- Logowanie do systemu.
- Zmiana języka interfejsu w czasie rzeczywistym (PL/EN).

B. Zalogowany Klient (Właściciel):

- **Zarządzanie zwierzętami:** Przeglądanie listy wyłącznie własnych zwierząt (filtrowanie po ID użytkownika).
- **CRUD:** Dodawanie, edycja i usuwanie własnych zwierząt.
- **Umawianie wizyt:** Formularz z blokadą dat przeszłych, wyborem lekarza i usługi z bazy.
- **Podgląd wizyt:** Widok zaplanowanych wizyt dla swoich zwierząt.

C. Weterynarz (Personel):

- Wszystkie uprawnienia podstawowe.
- **Globalny dostęp:** Dostęp do specjalnej zakładki "Wszyscy Pacjenci", wyświetlającej pełną bazę zwierząt w klinice (niezależnie od właściciela).
- **Weryfikacja danych:** Możliwość podglądu i usuwania dowolnego zwierzęcia w systemie.

5. Struktura Bazy Danych

Projekt wykorzystuje relacyjną bazę danych z 5 tabelami, spełniając wymóg relacji wiele-do-wielu:

1. **Users:** Przechowuje dane autoryzacyjne (email, hash hasła, imię, nazwisko, rola).
2. **Animal:** Dane pacjentów (klucz obcy do Users - właściciela).
3. **Service:** Słownik usług weterynaryjnych z aktualnymi cenami.
4. **Appointment:** Tabela wizyt (data, status, klucze obce do Animal i Users - lekarza).
5. **Appointment_Service:** Tabela łącząca (Wiele-do-Wielu).
 - *Kluczowa funkcjonalność:* Tabela ta zapisuje historyczną cenę usługi w momencie umawiania wizyty. Dzięki temu zmiana cennika w przyszłości nie wpływa na ceny wizyt już umówionych/odbytych.

6. Spełnienie Wymagań Dodatkowych

W projekcie zrealizowano następujące zaawansowane funkcjonalności:

- **Pełna Internacjonalizacja (i18n):** Cały interfejs (menu, formularze, komunikaty błędów, tabele) jest tłumaczony dynamicznie bez przeladowania strony.
- **Zabezpieczenia logiczne:** Blokada możliwości umawiania wizyt w przeszłości (walidacja frontend + backend).
- **Routing chroniony:** Niezalogowany użytkownik nie wejdzie na podstrony wewnętrzne, a Klient nie wejdzie na podstrony Weterynarza.
- **Obsługa błędów:** System informuje użytkownika o błędach sieci, zajętych emailu czy błędnym hasle poprzez komunikaty alert oraz w konsoli.